

PYTHON • PART 4

Object-Oriented Programming in Python

Classes, Objects, Constructors, Encapsulation, Inheritance,
Abstraction & Polymorphism — Theory + Hands-on Code

Instructor: Saif

AI/ML Engineer | Full-Stack Developer | Python Instructor

What We'll Cover Today

01

Why OOP?

Limits of lists & dictionaries

02

Classes & Objects

Blueprints and real instances

03

Constructors & Attributes

`__init__`, class vs instance data

04

Methods

Instance, class & static methods

05

Encapsulation

Public, protected, private + getters/setters

06

Inheritance

Single, multi-level & multiple

07

Abstraction

Abstract classes & methods

08

Polymorphism

Method overriding & duck typing

Why Do We Need OOP?

Imagine storing student records — each with a name, parent's name, address, graduation year, CGPA — plus behaviours like fee payment and scholarship checks.

✗ Without OOP (lists / dicts)

- Too many separate lists to track — one per property, per student
- Dictionary keys re-defined again and again for every student
- No clean way to attach behaviour (methods) to the data
- Hard to scale or maintain as the program grows

✓ With OOP (classes & objects)

- One class bundles all properties (attributes) together
- Behaviours (methods) live right next to the data they act on
- Create unlimited objects from one reusable blueprint
- Modular, reusable, and easier to maintain code

What Is Object-Oriented Programming?

A programming paradigm that organizes code into objects — bundles of data (attributes) and behaviour (methods) — modelling software the way we think about real-world entities.



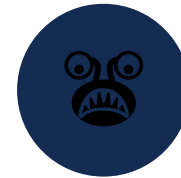
User



Car



Bank Account



Game Enemy

In Python, OOP is built around **classes** — blueprints for creating **objects**. It's not compulsory to always use OOP, but it helps a lot as programs grow in size and complexity.

Classes & Objects

Class

A blueprint or template for creating objects — like a recipe. It describes what an object will look like (attributes) and what it can do (methods), but it is not the object itself.

Object

A realization (instance) of a class — the actual thing created from the blueprint. We can create as many independent objects as we want from one class.

Note: We use the dot operator (.) to access properties and methods of an object, e.g. **car1.brand**

```
class Car:
    brand = "Toyota"

car1 = Car()
car2 = Car()
print(car1.brand)    # Toyota
print(car2.brand)    # Toyota
```

COMPARE

Class vs. Object

Class	Object
A blueprint / template	A concrete instance of a class
Does not exist in memory until instantiated	Contains actual data & occupies memory
One class can create any number of objects	Each object is independent

How to Create a Class — Step by Step

① Define the class

```
class Car:
    def __init__(self, brand, year):
        self.brand = brand
        self.year = year
```

- 1 **class** — keyword that starts a class definition
- 2 **Car** — the class name — use PascalCase (CarName style)
- 3 **:** — colon opens the class body (must be indented below)
- 4 **def __init__(self, ...)** — constructor — runs automatically on creation
- 5 **self.brand = brand** — stores the value as an attribute on the object

② Create an object from it

```
my_car = Car("Toyota", 2022)

print(my_car.brand) # Toyota
print(my_car.year) # 2022
```

- 1 **my_car** — any variable name you choose for the new object
- 2 **Car(...)** — calling the class like a function creates an instance
- 3 **"Toyota", 2022** — these get passed into `__init__` as brand, year
- 4 **my_car.brand** — dot operator reads the attribute off the object

How to Define & Call a Method

① Define the method inside the class

```
class Car:
    def __init__(self, brand):
        self.brand = brand

    def honk(self, times):
        print("Beep! " * times)
```

- 1 **def** — keyword that starts any function/method definition
- 2 **honk** — the method name — use snake_case (lower_case_words)
- 3 **(self, times)** — self is required first; times is a normal parameter
- 4 **:** — colon opens the method body (indent one level further)

② Call the method on an object

```
my_car = Car("Honda")
my_car.honk(3)
# Beep! Beep! Beep!
```

- 1 **my_car.honk(3)** — dot operator calls honk() on my_car
- 2 **self** — Python passes my_car into self automatically
- 3 **3** — this becomes the value of the 'times' parameter

Rule of thumb: you never pass a value for self yourself — Python fills it in with the object before the dot, every single time.

The `__init__` Constructor

A constructor is a special method used to initialize a newly created object. It doesn't create the object itself — it sets up the object with initial values. Python calls `__init__()` automatically whenever an object is created.

```
class Student:
    def __init__(self, name):
        self.name = name

stu1 = Student("Rahul")
stu2 = Student("Harshita")
print(stu1.name, stu2.name)
# Rahul Harshita
```

`self` refers to the specific instance calling the method. Python passes it automatically — we never pass it ourselves.

⚠ Good to know

Python doesn't support constructor overloading like Java/C++. If a class has multiple `__init__` definitions, only the last one written is used.

Types of Constructors

1. Default Constructor

Takes no parameters except self. Every object gets the same setup.

```
class Student:  
    def __init__(self):  
        print("created")
```

2. Parameterized Constructor

Takes parameters to initialize unique values for each object.

```
class Student:  
    def __init__(self, name):  
        self.name = name
```

Practical Tip

Use a parameterized constructor whenever objects need different starting data (e.g. each Student needs their own name & roll number). Use a default constructor only when there's truly nothing to customize.

Build a Class From Scratch (1/2)

Let's build a BankAccount class one piece at a time — the same order you'll follow in your own code.

1

Start with an empty class

```
class BankAccount:  
    pass
```

pass is a placeholder — it means "do nothing for now" so the class runs without errors while it's still empty.

2

Add a constructor + attributes

```
class BankAccount:  
    def __init__(self, owner, balance):  
        self.owner = owner  
        self.balance = balance
```

Replace pass with `__init__`. Every new BankAccount now needs an owner and a starting balance, stored as attributes via self.

Build a Class From Scratch (2/2)

3

Add a method (behaviour)

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
```

deposit() is indented at the same level as __init__ — both belong to the class. It updates the object's own balance.

4

Create an object & use it

```
acc = BankAccount("Ali", 1000)
acc.deposit(500)
print(acc.balance)
# 1500
```

acc is a real BankAccount object. Calling acc.deposit(500) runs the method with self = acc, so it changes acc's own balance.

Attributes: Class vs. Instance

Class Attributes

- Belong to the class itself — shared by ALL objects
- Defined outside any method, directly in the class body

```
class Student:
    college = "ABC college"

stu1 = Student()
print(stu1.college)
print(Student.college)
```

Instance Attributes

- Belong individually to each object — its own copy
- Defined inside `__init__()` using `self.attribute`

```
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self.gpa = gpa

stu1 = Student("Rahul", 8.7)
```

Instance, Class & Static Methods

1

Instance Method

Takes self as first argument. Can access both instance & class attributes.

```
def display(self):  
    print(self.name)
```

2

Class Method

@classmethod decorator. Takes cls (the class) as first argument. Works with class-level data.

```
@classmethod  
def change_school(cls, n):  
    cls.school_name = n
```

3

Static Method

@staticmethod decorator. Takes neither self nor cls. Grouped in the class for logical reasons.

```
@staticmethod  
def add(a, b):  
    return a + b
```

Encapsulation

Bundling data (variables) and methods (functions) that operate on that data into a single unit — a class — while controlling access to protect it from accidental or unauthorized modification.

Python's 3 Access Levels

Public

`name`

Accessible everywhere — normal variables.

Protected

`_age`

Single underscore. Accessible, but 'don't touch unless needed' — meant for internal/inheritance use.

Private

`__balance`

Double underscore. Name-mangled to `_ClassName__var` — cannot be accessed directly from outside.

Access Modifiers in Action

```
class Student:
    def __init__(self, name):
        self.name = name          # public

class Person:
    def __init__(self):
        self._age = 20           # protected

class Bank:
    def __init__(self, balance):
        self.__balance = balance # private
```

✓ Allowed

```
s.name
p._age  (works, but avoid)
```

✗ Blocked

```
b.__balance
# AttributeError
```

To access a private variable from outside, use its name-mangled form: **b._Bank__balance**. Python uses naming conventions, not strict enforcement (unlike Java/C++).

Getters & Setters

When a variable is made private, we use methods to safely read it (getter) or update it (setter) — instead of touching the private field directly.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary        # private

    def get_salary(self):              # getter
        return self.__salary

    def set_salary(self, new_salary): # setter
        self.__salary = new_salary

e = Employee(50000)
print(e.get_salary())    # 50000
e.set_salary(60000)
```



Getter

Reads the private value safely, without exposing the field itself.

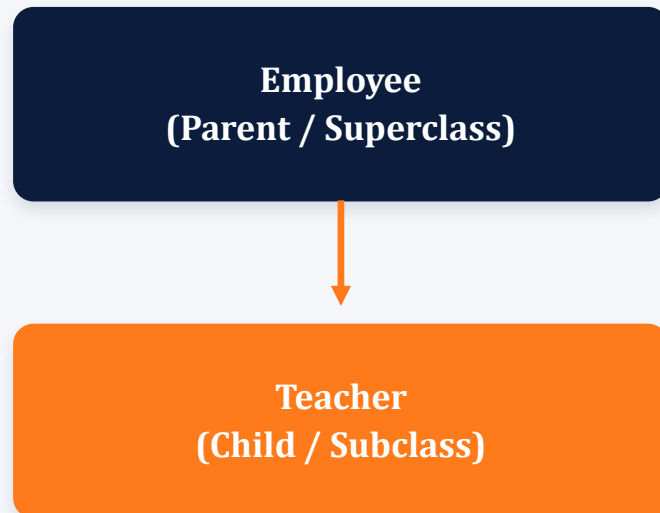


Setter

Updates the value — can add validation before changing it.

Inheritance

One class (child) acquires the properties and behaviours of another class (parent) — enabling code reuse, extensibility, cleaner design, and polymorphism.



```
class Employee:      # parent
    start_time = "9AM"
    end_time = "5PM"

class Teacher(Employee): # child
    def __init__(self, subject):
        self.subject = subject

t1 = Teacher("Data Science")
print(t1.subject, t1.start_time)
```

Types of Inheritance (1/2)

Single Inheritance

A child class inherits from exactly one parent class.

```
class Parent:
    def display(self):
        print("Parent class")

class Child(Parent):
    pass

c = Child()
c.display() # Parent class
```

Multi-level Inheritance

Grandparent → Parent → Child — each level builds on the one before.

```
class Employee:
    start_time = "9AM"

class AdminStaff(Employee):
    def __init__(self, role):
        self.role = role

class Accountant(AdminStaff):
    def __init__(self, salary, role):
        super().__init__(role)
        self.salary = salary
```

Types of Inheritance (2/2)

Multiple Inheritance — a child class inherits from more than one parent class at the same time.

```
class Teacher:
    def __init__(self, salary):
        self.salary = salary

class Student:
    def __init__(self, gpa):
        self.gpa = gpa

class TA(Teacher, Student):
    def __init__(self, name, salary, gpa):
        super().__init__(salary)    # Teacher init
        Student.__init__(self, gpa) # Student init
        self.name = name

ta = TA("Rahul", 50000, 7.5)
print(ta.name, ta.salary, ta.gpa)
```

super()

Calls the parent class's method from inside the child class — most commonly used to run the parent's `__init__`.

With multiple inheritance, `super()` only reaches the first parent — call other parents explicitly by name, as shown for Student above.

How to Inherit a Class — Step by Step

① Write the child's class line

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    pass
```

1 **class Dog(Animal):** — put the parent's name in parentheses after the child's

2 **pass** — Dog already works — it inherits `__init__` from Animal

② Add child-only behaviour with `super()`

```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

d = Dog("Rex", "Labrador")
print(d.name, d.breed)
```

1 **super().__init__(name)** — runs the parent's constructor first, for free

2 **self.breed = breed** — then add whatever is new for the child class

Abstraction

Hiding unnecessary implementation details and showing only the essential features. When you press a car's brakes, it stops — you don't need to know how the hydraulics work.

Abstract Class

- Cannot be instantiated directly
- Can contain normal + abstract methods
- Acts as a blueprint that children must complete

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):  
    @abstractmethod  
    def sound(self):  
        pass
```

```
from abc import ABC, abstractmethod  
  
class Animal(ABC):  
    @abstractmethod  
    def make_sound(self):  
        pass  
  
class Lion(Animal):  
    def make_sound(self):  
        print("Roar!")  
  
class Cow(Animal):  
    def make_sound(self):  
        print("Moo!")  
  
Lion().make_sound() # Roar!  
Cow().make_sound()  # Moo!
```

Polymorphism: Method Overriding

"Poly" = many, "morph" = forms. The same method name behaves differently depending on the object calling it. Even Python's + operator is polymorphic: 1+2 adds, while "1"+"2" concatenates.

```
class Animal:
    def sound(self):
        print("Some generic sound")

class Dog(Animal):
    def sound(self):          # overrides parent
        print("Bark")

a = Animal()
dog = Dog()
a.sound()    # Some generic sound
dog.sound()  # Bark
```

Method Overriding

A child class provides its own version of a method that already exists in the parent (same name).

Runtime polymorphism — the child's method takes precedence over the parent's.

Polymorphism: Duck Typing

"If it looks like a duck and quacks like a duck, it must be a duck." Python doesn't check an object's type before calling a method — only whether the method exists.

```
class Dog:
    def speak(self):
        print("Bark")

class Cat:
    def speak(self):
        print("Meow")

class Robot:
    def speak(self):
        print("Beep Boop")

def make_it_speak(entity):
    entity.speak() # type doesn't matter

for e in [Dog(), Cat(), Robot()]:
    make_it_speak(e)
```



Dog

→ Bark



Cat

→ Meow



Robot

→ Beep Boop

PRACTICE

Try It Yourself: Build a Book Class

Open your editor and follow these steps — this single exercise touches attributes, a constructor, a method, and encapsulation.

1 Create the class

Write `class Book:` then add `__init__(self, title, author, pages)` and store all three as self attributes.

2 Add a private attribute

Add `self.__is_borrowed = False` inside `__init__` — this tracks state that outside code shouldn't set directly.

3 Add a method

Write `def borrow(self):` that sets `self.__is_borrowed = True` and prints a confirmation message.

4 Add a getter

Write `def is_borrowed(self):` that returns `self.__is_borrowed`, so others can check the status safely.

5 Test it

Create `b = Book(...)` , call `b.borrow()` , then print `b.is_borrowed()` to confirm it now returns True.

The 4 Pillars of OOP



Encapsulation

Bundle data & methods; control access with public / protected / private.



Abstraction

Hide implementation details; expose only what's essential via abstract classes.



Inheritance

Reuse a parent's attributes & methods in a child class (single, multi-level, multiple).



Polymorphism

One method name, many behaviours — overriding & duck typing.

Keep Learning & Keep Exploring!

Practice each pillar with your own mini-project — a Library System or a Zoo simulation covers all four beautifully.

Saif

AI/ML Engineer | Full-Stack Developer | Python Instructor

